

History-driven Compiler Fuzzing via Assembling and Scheduling Bug-triggering Code Segments

Zhenye Fan, Guixin Ye*, Tianmin Hu, Zhanyong Tang*
Northwest University, China
{fanzhenye, hutianmin}@stumail.nwu.edu.cn, {gxye, zytang}@nwu.edu.cn

2024 IEEE 35th International Symposium on Software Reliability Engineering (ISSRE) | 979-8-3503-5388-4/24/$31.00 ©2024 IEEE | DOI: 10.1109/ISSRE62328.2024.00040

***Abstract*—History-driven testing techniques have been proven to be an effective method for detecting compiler bugs. It employs fuzzing history (e.g., historical test cases or historical execution information) to guide to generate valid test cases. However, prior methods either have an inefficient capability in synthesizing bug-triggering test cases or suffer from a plateau of code coverage, causing a low bug-exposing ability. This paper presents ASMFuzz, another history-driven compiler testing framework by applying a multi-metric hybrid scheduling strategy. Specifically, ASMFuzz first extracts the bug-triggering code segments from the historical test cases that triggered bugs. The extracted bug-triggering code segments are then used to assemble new test cases. To ensure the correctness of the newly synthesized test cases, ASMFuzz always selects the segments with code context dependencies for assembly. Duration assembly, the ingredients to be assembled are determined based on multiple feedback metrics (e.g., anomalous behaviors and code coverage). To do so, we proposed a multi-metric hybrid scheduling scheme to select optimal code segments in each testing iteration. This contributes to continuously covering deep code branches of compiler duration whole testing process, avoiding getting stuck in the plateau of code coverage. We evaluated ASMFuzz on three mainstream JVMs including OpenJ9, HotSpot, and GraalVM involving six JDK versions. Within a 72-hour concurrent test run, ASMFuzz exposed 16 previously unknown unique bugs, of which 11 have been confirmed by the developers. We also compared ASMFuzz to four prior state-of-the-art fuzzers. ASMFuzz uncovers 1.6~2.2× more bugs than comparative baselines.**

***Index Terms*—Compiler Fuzzing, Software Testing, JVM, Bug Finding**

I. Introduction

Java Virtual Machine (JVM) forms the core environment for executing Java applications, implementing an abstract computational platform through software technology. Java source code must first be compiled into `.class` files, and then JVM can load and execute these files. JVM, with its tailored versions for diverse operating systems, equips an interpreter that ensures the cross-platform execution of Java programs upon installation, which realizes the Java language "write once, run everywhere" design concept. There are several JVM implementations available in the market, including HotSpot [1] and GraalVM [2] from Oracle, OpenJ9 [3] and JikesRVM [4] from IBM, Zulu [5] and Zing [6] from Azul. Even though these JVM versions have been optimized and maintained for a long time, these JVM versions may still present varying degrees of defects and vulnerabilities. Given the complexity inherent in the design of the JVM, the processes of maintenance and repair are not only costly but also carry the risk of introducing new defects in the course of rectification [7; 8]. Despite the maturity of JVM technology, potential software defects still pose serious challenges [9; 10]. Ensuring the stability and security of the runtime environment is crucial for applications relying on JVM.

To detect the bugs of JVM, researchers have proposed several promising fuzzing techniques [11; 12; 13; 14; 15; 16; 17]. Classfuzz [11] designs a series of mutation rules, such as replacing or deleting code fields, changing variable modifiers or types, etc. This approach is highly random, and although it is effective in increasing the diversity of test cases, it can easily lead to program syntax errors. Classming [12] avoids the uncertainty of random mutation by inserting mutating instructions into the bytecode file to change the control and data flow of the program, thus enriching the diversity of test cases in terms of control and data flow. JavaTailor [13] employs code segments extracted from historical test cases and randomly inserts them into seed programs to assemble new test cases. This random assembling strategy can generate syntax-correct test cases, but it is hard to make full use of the bug-triggering code segments in the historical test cases, resulting in a huge generation space for triggering deep code branches. Although VECT [15] enhances the generation of historical test cases by vectorizing program ingredients, the substantial size of the ingredient space could lead to components with analogous semantics not being adequately prioritized within the clustering process according to their bug-triggering capabilities. LERE [18] clarifies the elements of bug reports into three categories: programs, comments, and others. It utilizes programs directly as test cases, but this also limits the diversity of test cases. In fact, throughout the iterative process of software versioning, it is customary for developers to update regression tests after modifying the source code, but such regression testing is typically restricted to checking the updated code against expected outcomes. Manually writing more complex test cases incurs higher time costs. By combining patch information with historical test cases, it becomes feasible to swiftly identify the pertinent code snippet responsible for triggering errors. Concurrently, this approach enables the rapid generation of grammatically correct, bug-triggering test cases.

Although generating test cases that can trigger bugs is an important part of fuzzing, the initial stage of synthesizing test cases lacks complex semantics. In practice, it is often necessary to iteratively assemble seed files to expand the coverage progressively. Notably, guided scheduling metrics significantly

*Guixin Ye and Zhanyong Tang is the corresponding author.

2332-6549/24/$31.00 ©2024 IEEE
DOI 10.1109/ISSRE62328.2024.00040

Authorized licensed use limited to: NORTHWESTERN POLYTECHNICAL UNIVERSITY. Downloaded on December 04,2024 at 08:55:10 UTC from IEEE Xplore. Restrictions apply.

enhance both the assembly and testing efficiency throughout the iterative process. The state-of-the-art (SOTA) tools have demonstrated the effectiveness of scheduling strategies. SJ-Fuzz [14] uses discrepancy-guided and edit-distance scheduling to optimize seed and mutation operators, while VECT schedules by classifying components for aggregation. These strategies guided by consistency and discrepancy information effectively improve the efficiency of testing. However, the feedback used by existing JVM fuzzing tools is still relatively singular. Even after multiple rounds of iterations, the coverage of JVM may still be limited [19]. In this context, researchers have explored the diverse possibilities of multi-metric feedback.

AFL [20] implements a single feedback mechanism by increasing the number of metrics, specifically updating the feedback using the product of execution time and test case length. However, this singular approach to feedback often reach a coverage plateau within a short period of testing runs. This will fall into a state of convergence and be hard to coverage new code branches. A AFL-based mutational fuzzer [21] parses the code into templates and fragments, and randomly splices them together to generate syntactically well-formed programs, but this is based on a randomized strategy and it is difficult to guarantee the syntactic correctness of the Java program. Although Cerebro [22] and MobFuzz [23] use a non-dominated sorting algorithm for multiple feedback optimization. Despite their advancements, these tools still fall short in integrating dynamically scheduled metrics. Additionally, they often fail to adequately consider the performance impact that the collection and analysis of these metrics can have on the overall efficiency of the testing process.

In this work, we introduce ASMFUZZ, a compiler fuzzing framework by combining bug-triggering code segments assembling and scheduling strategies. ASMFUZZ is specifically designed to test historically more error-prone compiler components using the segments that triggered the historical bugs. Therefore, ASMFUZZ obtains high-quality test cases that are related to historical bugs. By designing an adaptive scheduling strategy with multiple feedback metrics including anomalous behavior and coverage information, ASMFUZZ can efficiently utilize bug-triggering code segments. To further solve the problem of cumbersome and complex coverage information, we quantify it through the concept of information entropy. This not only provides more information about functional modules and internal paths for the feedback strategy but also discovers more bug-triggering paths through anomalous behavior feedback.

We implemented a prototype of ASMFUZZ. To evaluate ASMFUZZ, we conducted extensive experiments by applying it to three mainstream JVMs: HotSpot, OpenJ9, and GraalVM. Within 72 hours of automated test runs, ASMFUZZ exposed 16 previously unknown bugs in all three tested JVMs. Of all 16 reported bugs, 11 have been confirmed by the corresponding JVM developers. We helped developers fix 2 bugs by submitting PR to related repositories hosted on GitHub. We also compared ASMFUZZ to four SOTA fuzzers [13; 23; 15; 14]. The comparison results show that ASMFUZZ is effective in generating bug-triggering test cases by assembling historical test programs, exposing more bugs than the comparative fuzzers within the same testing time.

```
1   import sun.misc.Unsafe;
2   import java.lang. reflect .Field;
3   import java.nio.*;
4   import sun.nio.ch. DirectBuffer;
5
6   public class MyJVMTest {
7       String getAll() throws Exception {
8         Field theUnsafe = Unsafe.class.getDeclaredField("theUnsafe");
9         theUnsafe. setAccessible(true);
10  ++    Unsafe unsafe = (Unsafe)theUnsafe.get(null);
11        /*--Bug-triggering Code Segment--*/
12  +     ByteBuffer b = ByteBuffer. allocateDirect (0);
13  +     long addr = ((DirectBuffer)b).address();
14  +      int value = 10;
15  +      int len = (int)Math.pow(2,7);
16        /*--Bug-triggering Code Segment--*/
17  +     for (int i = 0; i < len; i++){
18  +       unsafe.putByte(null,addr+i,(byte)value);
19  +         addr+i;
20  +         unsafe.getByte(addr+i);
21  +      }
22        return null;
23      }
24      public static void main(String[] args) throws Exception {
25        System.out. println (new MyJVMTest().getAll());
26      }
27  }
```

Listing 1: ASMFUZZ-generated test case that exposes a new bug of OpenJ9.

To summarize, this work makes the following contributions:

- We propose ASMFUZZ, another history-driven compiler fuzzing framework that combines code assembling and multi-metric hybrid scheduling strategies to generate bug-triggering test cases, posing a feasible solution for breaking through the plateau of code coverage.
- We implement a prototype of ASMFUZZ[1], locate bug-triggering code segments through JavaParser [24] and PR, and implement scheduling policies through LinUCB [25] for replication and further optimization for future use.
- We conducted extensive experiments to evaluate the effectiveness of ASMFUZZ. It identified 16 confirmed bugs, including 12 OpenJ9 bugs, 2 GraalVM bugs, and 2 HotSpot bugs. 2 of all reported bugs have been fixed by contributing PRs to the source code of the corresponding JVM repositories.

II. Background and Motivation

A. History-driven Compiler Testing and Challenges

History-driven testing techniques have been considered an effective approach for compiler testing [13; 26]. The insight is that bug-triggering test cases can be synthesized by assembling code snippets extracted from historical test programs. This idea is motivated by the fact that many bugs are caused by the incomplete or incorrect repair of historical bugs [7; 13]. Accordingly, researchers have proposed a series of fuzzers for compiler testing [27; 28; 29; 30; 13; 31; 32; 33; 34; 35; 36], exposing hundreds of bugs or vulnerabilities for compilers. While promising, prior methods are inefficient in synthesizing bug-triggering test cases. This is because all of them adopt random strategies to assemble code snippets extracted from historical test programs into the syntax-correct test cases, failing

[1]https://zenodo.org/records/12705293

Authorized licensed use limited to: NORTHWESTERN POLYTECHNICAL UNIVERSITY. Downloaded on December 04,2024 at 08:55:10 UTC from IEEE Xplore. Restrictions apply.

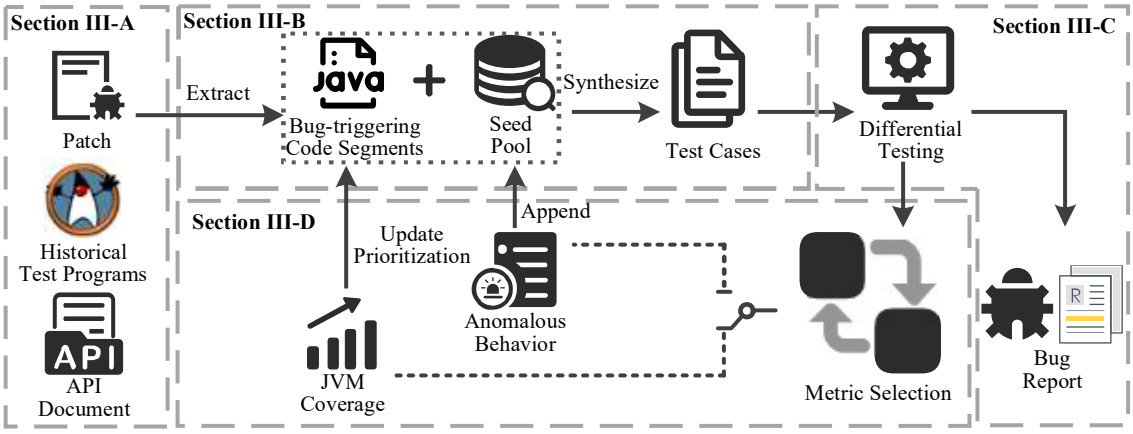


Figure 1: The overview of ASMFUZZ. It utilizes code segments assembling and multi-metrics scheduling strategy for history-driven compiler fuzzing.

to make full use of the code snippets in the historical test programs that are relevant to triggering bugs. A recent study indicates that only a small part of code in a test program is relevant to triggering the bugs [7]. This means that prior randomly assembled test cases in prior work have a low bug-triggering capability. Thus, the first challenge for history-driven testing is ***how to assemble the historical test programs into bug-triggering test cases?***

To address the above challenge, a feasible solution is to employ scheduling-based testing techniques to choose bug-exposing code segments for generating valid test cases. Prior scheduling-based efforts can be divided into two categories: (1) coverage-based methods [20; 37] that apply code coverage to guide seed selection; (2) entropy-directed methods [38; 39; 40] that utilize entropy calculated by historical testing information (e.g., Number of executions or output uniqueness) to guide to choose the optimal mutators. While prior scheduling-based methods pose improvements in discovering new code branches, they often reach a coverage plateau after a short testing period [19]. This means prior methods turn to a convergence state and hard to discover new code branches, resulting in a limited bug-exposing capability. Thus, another challenge for history-driven testing is ***how to break through the plateau of code coverage and continuously discover new code branches within the duration of testing?***

B. Motivation Example

Listing 1 provides a showcase for an `OpenJ9` bug, which reveals an internal optimization defects related to memory buffer allocation. The bug is triggered by the `putByte` method (line 18). Specifically, the method `putByte` aims to write the variable `value` to the buffer `addr` whose length is 0 allocated by the method `allocateDirect` (line 12). However, `OpenJ9` does not implement optimization for the write operation with `Unsafe` type. Consequently, `OpenJ9` crashes when executing the test cases while other JVMs like HotSpot successfully executes without throwing any anomalous behaviors due to pre-allocating a temporary space.

The test case is generated by iteratively assembling two bug-triggering code segments that are collected from historical test programs. Specifically, the first code segment at line 11 is a variable statement that is extracted from the historical test file `CopyMemory.java` [41]; the other code segment at lines 17-21 is a `for` statement that is cropped from the historical test file `Test7190310-unsafe.java` [42].

In the synthesis process, the seeds in the seed pool are first combined with the bug-triggering code segment, `unsafe`(line 10) is introduced. Since `unsafe` called `unsafe.getInt` because the parameter was too large resulting in anomalous crash, the test case containing *unsafe* was put into the seed pool. In another round of synthesis, the synthesis process picks the `For` block based on the priority of the segment. The insertion requires the introduction of undefined variables `addr`,`len`,`value` based on contextual analysis (lines 13-15). Finally, scanning the newly introduced code (line 13) revealed the undefined variable `ByteBuffer b` and introduced it. The syntactically correct test case was obtained after several rounds of assembly. This inconsistency looks like simple code assembly, but current mainstream tools are not yet able to expose similar anomalies. This is because only multiple updates to the seed pool can cause multiple bug-triggering code segments to be synthesized in the test case. This suggests that it is very difficult to select the segments that can trigger a bug in a huge component space.

III. DESIGN

In this work, we propose a novel testing technique, called ASMFUZZ, which aims to use patch information to locate bug-triggering code segments to improve the quality of assembled test cases. Specifically, when extracting historical test cases, ASMFUZZ collects bug-fixing patches and extracts bug-related code segments, effectively avoiding the redundant segments brought about by complete extraction. To achieve continuous and efficient testing, we designed a multi-metric hybrid scheduling strategy, which combines the statistical time of the feedback metrics to guide the code segment schedule.

Authorized licensed use limited to: NORTHWESTERN POLYTECHNICAL UNIVERSITY. Downloaded on December 04,2024 at 08:55:10 UTC from IEEE Xplore. Restrictions apply.

```
209     ......  // Other Sections of Historical Test Cases
210   public final List<String> getInputArguments() {
211         checkMonitorPermission();
212   +     return Collections.unmodifiableList(
213   +         ManagementUtils.convertStringArrayToList(VM.getVMArgs()));}
214     ......  // Other Sections of Historical Test Cases
```

Listing 2: Bug-triggering code segments Associated with PR#17699.

As illustrated in Figure 1, we first locate and parse the extracted bug-triggering code segments (Section III-A) and construct a seed pool based on previous work [26]. Then, we assemble test cases using a greedy strategy (Section III-B). Differential testing not only checks whether the test program reveals bugs or not, but also provides execution information for the scheduler (Section III-C). Finally, ASMFUZZ designs two metrics based on anomalous behavior and code coverage, to guide the code segment schedule for further testing iteration (Section III-D).

A. Bug-triggering Code Segments Collection

We first developed an HTML crawler to collect all the pull requests (RPs) from GitHub repositories of the test JVMs and obtain all test programs by matching keywords `test` and `.java`. Then, we used JavaParser [24], a parser for Java programs, to convert test programs into abstract syntax trees (ASTs). We focus on code segments related to codes that have been added, i.e., code lines marked with a `+`. After extracting these bug-triggering code segments, we also identify all variables declared and used in those segments, which can be conveniently accessed from AST nodes. This information will be utilized in Section III-B.

In software development, a patch refers to changes or updates made to the code repository, primarily aimed at fixing bugs or adding new features. These modifications may involve the code, configuration files, or documentation. A Pull Request (PR) is a standardized collaborative workflow that allows developers to merge their changes into the main code repository. A typical PR process not only involves updates to the source code but also to the corresponding test cases.

In the JVM repository, we observed that PRs labeled with bug tags often involve crucial functional modules such as hotspot, core-libs, and tools. ASMFUZZ utilizes the information in these PRs to extract patches. Specifically, when keywords associated with `bug` tags are found in a PR, ASMFUZZ locates changes to variables and code blocks in the test cases and extracts them.

B. Test Case Synthesis

To generate a new test case, ASMFUZZ builds a seed pool by using the generator of COMFUZZ [26], and selects a bug-triggering code segment and a seed program, and then inserts the segment into the seed program.

The detailed process is described in Algorithm 1. Firstly, ASMFUZZ derives variables of the seed file SF (line 1), selects a bug-triggering code segment (line 2) and obtains the variables of the selected segment (line 3). Subsequently,

Algorithm 1: Greedy Synthesis Algorithm

Input: BT: Bug-triggering Segments, SF: Seed File
Output: $Testcase_list$: Test Cases

```
1  SfVariables ← GetVariables(SF)
2  bt ← SelectBTData(BT)
3  RelevantVariables ← bt.GetRelevantVariables()
4  while RelevantVariables is not empty do
5      relevantInfo ← RelevantVariables.Pop()
6      if CheckIfNeedGenerate(relevantInfo, SfVariables) then
7          if NeedMatchBTPool(relevantInfo) then
8              bt_new ← GetVariables(SF)
9              RelevantVariables.Push(bt_new)
10         else
11             context ← GenerateFromAPI(relevantInfo)
12             bt_list ← Combine(context, bt)
13 AnalyseVariableConflicts(bt_list, SeedFile)
14 Testcases ← GreedyInsert(bt_list, SeedFile)
15 return Testcases
```

ASMFUZZ checks whether there is a need to generate new dependencies (line 6). If new dependencies are required, there are two methods to handle them: One method retrieves other bug-triggering code segments, where the code block context is matched with bt (line 7), searches for variables that match the context information (line 8) and add their dependency information to `RelevantVariables` (line 9); another method directly search from the `API` standard information library (line 11) and randomly generating values of the corresponding type (line 12). The process of code generation continues in subsequent iterations until the `RelevantVariables` list is emptied (line 4). Subsequently, ASMFUZZ solves the variable name conflict between the seed file and the code segment (line 13). If the variable name already exists, ASMFUZZ modifies it into a new one. Finally, ASMFUZZ inserts the code segment into the seed file greedly (line 14). This is done by statistically counting the number of required variables before each seed insertion point, and the insertion point with the maximum variables will be selected.

C. Differential Testing

In the differential testing phase, we select two versions (11 and 17) of Java compilers from three vendors (OpenJ9, GraalVM, and HotSpot). On the one hand, we can horizontally compare the compiler developed by different teams to assess the differences in performance and functionality more comprehensively, and on the other hand, we can also vertically compare the performance on two different versions of the JDK, to effectively assess the functionality and performance differences caused by the iterative updating of versions. At the same time, we use GCOV [43] to instrument the underlying CPP files for OpenJDK and use LCOV [44] to collect the detailed coverage of the underlying source code. The coverage results are stored in `.gcno` and `.gcda` files.

After sending a test case to several target compilers, ASMFUZZ obtains their return codes, outputs, and error messages. Based on this information, ASMFUZZ detects three types of anomalous behaviors: inconsistency, timeout, and crash. Notably, to reduce false positives, ASMFUZZ filters

Authorized licensed use limited to: NORTHWESTERN POLYTECHNICAL UNIVERSITY. Downloaded on December 04,2024 at 08:55:10 UTC from IEEE Xplore. Restrictions apply.

Table I: Metrics used in the scheduling

Name	Definition
A_a	Guidance metric information matrix
b_a	Guidance metric response vector
$\hat{\theta}_a$	Guidance metric estimation weight vector
$\mathcal{S}_a$	Guidance metric selection score

inconsistency due to threads, time, and random operations, thus reducing the cost of manual analysis.

For continuous and efficient testing, ASMFUZZ integrates two feedback mechanisms: anomalous behavior feedback and code coverage feedback. If the anomalous behavior feedback is activated, the test case will be added to the seed pool for further test case synthesis. If the code coverage feedback is activated, the system obtains the coverage information through the instrumented compiler and quantifies the coverage information using an improved information entropy formula to update the prioritization of the bug-triggering code segments.

D. Multi-metric Guided Scheduling

We design a multi-metric hybrid scheduling scheme in conjunction with `LinUCB` (Linear Upper Confidence Bound). This strategy not only reflects the execution of test cases more accurately but also combines the feedback time of the metrics to select the most appropriate feedback strategy at present. In particular, ASMFUZZ introduces bug-triggering code segments scheduling metric based on JVM coverage. If the anomalous feedback strategy enters the local optimal solution, the coverage information is quantified by information entropy, which improves the test case diversity in the feedback.

1) Guided Feedback Metric Selection: Our metric selection uses the contextual information in `LinUCB` to model the statistical time cost of applying feedback mechanics and make decisions in each round of metric selection. The `LinUCB` algorithm is a multi-armed bandit algorithm that combines the `UCB` strategy with a linear regression model. It is primarily used to balance exploration and utilization decisions based on experience. Specifically, anomalous behaviors of differential testing serve as important feedback that helps to explore new bugs in specific modules quickly, but it cannot increase the diversity of test cases. Other feedback metrics, such as code coverage, are selected by the scheduling algorithm to update the priority of the bug-triggering code segment, which leads to other key modules for testing.

For each metric feedback parameter α, ASMFUZZ initializes the feedback metrics as $A_a = I_d$ ($d \times d$ identity matrix) and $b_a = \mathbf{0}$ (d-dimensional zero vector). During each test iteration, begin by acquiring the context $\mathbf{x}_t$. Next, ASMFUZZ computes the estimated parameter vector $\hat{\theta}_a = A_a^{-1} b_a$ and the score metric:

$$\begin{aligned} \mathcal{S}_a = &\quad E[r_a|\mathbf{x}] + \alpha \mathrm{STD}(r_a|\mathbf{x}) \\ = &\quad \hat{\theta}_a \mathbf{x}_t^T + \alpha \sqrt{\mathbf{x}_t^T A_a^{-1} \mathbf{x}_t} \end{aligned} \tag{1}$$

Subsequently, ASMFUZZ selects the next feedback metric by the following formula:

$$a_t = \arg\min_a \mathcal{S}_a \tag{2}$$

Finally, ASMFUZZ concurrently update the context information matrix $A_{a_t} = A_{a_t} + \mathbf{x}_t \mathbf{x}_t^T$ and the corresponding vector $b_{a_t} = b_{a_t} + r_t \mathbf{x}_t$.

2) Anomalous Behavior Feedback: Fuzzing reveals potential defects by executing a large number of test cases. During this process, fuzzing usually triggers various anomalies such as crashes, errors, and performance degradation. These anomalies are represented by feedback in different formats. Anomaly messages can be extracted by Regular matching, which in turn can accurately determine the type of anomalies in test cases. In addition, analyzing these anomalous behaviors reveals the code segments that trigger defects and finds that different anomalies have different code coverage characteristics. Based on this anomaly feedback, normal test cases assembled from data with key features can be added to the seed pool, which can effectively introduce multiple patch history test fragments. This strategy helps to prioritize the discovery of other important defects in the same module, thus improving the efficiency and coverage of fuzzing.

3) Code Coverage Feedback: Using code coverage for test feedback consumes significant computational resources, but source code coverage information is critical for guiding test case synthesis. Ideally, coverage feedback should be selected after a module has been fully tested, rather than provided in real time. Feedback increases the probability of shifting the testing focus to other modules for more targeted testing. This approach optimizes resource usage while ensuring more comprehensive testing.

$$\begin{aligned} H(m) = &\quad -\sum_{i=1}^{C} p_i' \log(p_i') \\ = &\quad \log(\sum_{j=1}^{C} p_j) - \frac{\sum_{i=1}^{C} p_i \log(p_i)}{\sum_{j=1}^{C} p_j} \end{aligned} \tag{3}$$

The JVM interacts with the operating system through the Java Native Interface (JNI), where `CPP` files are called by Java files as native methods that are responsible for interacting with the operating system and important underlying functions such as JVM performance optimization. In practical calculation, due to the multidimensional complexity of coverage information, this paper employs the method to quantify the coverage of each line or branch of the JVM source code at a fine granularity after instrumentation for the above `CPP`. To establish a relationship between bug-triggering code segments and individual lines and branches of JVM source code, we first collect coverage data from the JVM source during the generation of test cases from bug-triggering code segments m.

To establish the relationship between the bug-triggering code segments and the source code of JVM, we first counted the code coverage of JVM during the execution of the generated test cases that contain bug-triggering code segments m. Assuming the total number of JVM source code lines and branches is C, the coverage frequency f_i for each line or branch i within each CPP file and its header files is recorded. Subsequently, the coverage frequency p_i for each line or branch i is calculated based on f_i and the total number C, and normalized using the formula $p_i' = p_i / \sum_{j=1}^{C} p_j$, resulting in the normalized

Authorized licensed use limited to: NORTHWESTERN POLYTECHNICAL UNIVERSITY. Downloaded on December 04,2024 at 08:55:10 UTC from IEEE Xplore. Restrictions apply.

Table II: Target JVMs tested in our evaluation

JVMs	JDK Version	Build Number	Release Date
OpenJ9	17	17.0.9+8	Apr. 2024
	11	11.0.17+8	Apr. 2024
GraalVm	17	17.0.9+9	Oct. 2023
	11	11.0.20+8	Jul. 2023
HotSpot	17	17.0.11+9	Apr. 2024
	11	11.0.23+9	Apr. 2024

frequency p'_i. Lastly, using Equation 3, the information entropy value $H(m)$ for the current source code coverage is calculated based on the obtained frequencies. This entropy value is then utilized to update the priority of bug-triggering code segments, guiding the fuzzer to select bug-triggering code segments with higher entropy values in subsequent iterations, thereby enhancing the effectiveness of detecting potential defects.

IV. Experimental Setup

In our experiments, we set out to investigate the following research questions:

- **RQ1:** How ASMFuzz perform in exposing previous unknown bugs for JVMs?
- **RQ2:** Is the synthesis of bug-triggering code segment effective for enhancing fuzzing techniques?
- **RQ3:** Does a multi-metric feedback scheduling approach improve the efficiency of fuzzing?

A. Benchmark

We deployed the latest versions of JDK11 and JDK17 JVMs from different development teams, including HotSpot, OpenJ9, and GraalVM, for differential fuzzing. Table II shows the specific detailed versions of these JVMs. In all our comparison experiments, we chose HotSpot as the target JVM for comparison. This is because it is the most widely used JVM and it was also used by the comparative baselines to evaluate code coverage rates. And evaluate metrics by counting syntactic correctness, and suspicious test cases.

We select four SOTAs for complete comparison. Two of them focus on scheduling, SJFuzz proposed the variational operator scheduling, and VECT utilized compositional selection as comparative benchmarks. Both of them perform well in existing testing methods. JavaTailor fully utilizes the extraction of historical test cases and retains the most seed programs, so it is the best tool for comparison of extraction methods. Since MobFuzz has no publicly available source code, we replicate the excellent feedback strategy described in its paper.

B. Data Preparation

Seeds. We adopt the seed pool construction method from the JVM part of COMFUZZ. It trains a code generator with a large amount of Java programs, which has also been shown in papers to provide semantically rich Java code seeds. Meanwhile, we exclude test programs that may reveal errors on ASMFuzz or fail to run successfully in the experiments to ensure the validity of the experiments and the accuracy of the results. Such a screening process contributes to improving the efficiency and quality of fuzzing.

Table III: ASMFuzz-exposed bugs.

JVMs	Submitted	Confirmed	Fixed
OpenJ9	11	6	1 (PR#17699)
GraalVM	2	2	0
HotSpot	3	3	1 (PR#2425)
All	16	11	2

Bug-triggering code segments. To construct the most essential bug-triggering code segments, we utilized JavaParser technology [24] to extract from the patch and establish an API standard information library. This library is a crucial reference for identifying context dependencies and generating context-relevant code. The code blocks can be categorized into six types: `For`, `While`, `For-each`, `Do-while`, `If-type` code blocks, and containing `API` call information. The process of synthesizing bug-triggering code segments during fuzzing is critical. We employ a greedy strategy to optimize the combination of seeds and bug-triggering code segments, which necessitates that segments include essential context information.

C. Environmental Platform

The prototype of ASMFuzz is implemented on a high-performance server powered by an Intel i7-14700K processor with a 3.40 GHz clock speed, 48 GB of RAM, and running Ubuntu 22.04. The system utilizes `MySQL` to manage four databases: the bug-triggering code segments pool, seed pool, API standard library, and differential testing process information. System operations and executions are managed using Python 3.8.10 and Java 11.0.17.

V. Results And Analysis

*1) **Detected Bugs (RQ1)**:* After 72 hours of testing, ASMFuzz detected 16 previously unknown bugs, of which 11 were confirmed by the developer, and 2 were fixed by submitting `PRs`. Table III shows the details of the confirmed/fixed bugs detected by ASMFuzz. We then use one of the detected unknown bugs as an example for further illustration.

Bug example. Listing 3 illustrates a test case that triggers a functional bug, where the `If` code blocks (lines 10-13) are performed to synthesize a new test case. It is worth noting that since `optionCount` and `rm` are undefined variables in the code block, and `rm` has been initialized (line 6), only the `optionCount` variable needs to be added (line 7) and `buf` variable has been introduced and successfully matched with the `buf` in the seed (line 2) to form this test case.

The logic of the test case triggering the API bug in OpenJ9 is as follows. The seed file of this test case is derived from the results of the previous iteration of anomaly feedback, and the new test case is synthesized again with bug-triggering code segments. One historical test program is `Indify.java` [45] that detects the historical bug [46]; the other is `InputArgument.java` [47] that tests the historical bug [48]. Note that we remove the extraneous conditions, i.e., *&& (con = parseMemberLookup(refKind, args))! = null* when assembling the code segment.

After manually analyzing, the function called *ManagementFactory.getPlatformMXBeans* (line 11), which was determined

Authorized licensed use limited to: NORTHWESTERN POLYTECHNICAL UNIVERSITY. Downloaded on December 04,2024 at 08:55:10 UTC from IEEE Xplore. Restrictions apply.

```
1   public class MyTest {
2       static byte[] buf = { 23, −118, −4, −81, 122, 48 };
3       static byte[] arg0 = { 80, −44, −101, 59, 62, −55 };
4       static int arg1 = 1;
5        ......
6       static RuntimeMXBean rm = ManagementFactory.
                getRuntimeMXBean();
7   +    static int optionCount = buf.size();
8       int write(byte[] arg0, int arg1) throws Exception {
9           ......
10  +        if (optionCount > 0){
11  +            List<String> options = rm.getInputArguments();
12  +            options.clear();
13  +        }
14              ......
15          int newPos = optionCount + arg1;
16          return newPos;
17      }
18      public static void main(String[] args) throws Exception{
19          System.out.println(new MyTest().write(arg0, arg1));
20      }
21  }
```

Listing 3: Test case that triggers an API functional bug.

as an exception by the system when obtaining `ThreadIds`, but due to the compliance with the false positive rule, a test case without error was put into the seed pool through the anomalous behavior feedback. By combining with the new bug-triggering code segments (lines 10-13), when calling `clear`, due to the design of `getInputArguments` in OpenJ9, the return is designed as a modifiable list, which leads to memory thread leakage and triggers a flaw in the interface of OpenJ9. Eventually, developers fixed this bug, and the description of PR is as follows[2].

> *When using* Collections.unmodifiableList *to wrap a list, it ensures that the returned list is read-only, effectively preventing any attempts to modify the list.*

Given the target seed and bug-triggering code segment, JavaTailor and VECT can also generate the test cases in Listing 3. However, they need to spend enough time to find the right seed and code segment. In contrast, ASMFuzz collects focused code segments and applies a multi-metrics guided scheduling scheme to shorten the generation time.

*2) **Effectiveness of bug-triggering code segment (RQ2)**:* In order to verify the validity of the bug-triggering code segments, we design four different synthesized components in this part: ❶Test cases based on seed files (SEED); ❷Test cases only insert variables in bug-triggering code segments (BT-Var); ❸Test cases only insert code blocks in bug-triggering code segments (BT-Blocks); ❹Test cases insert complete bug-triggering code segments (BT). After a 24-hour experiment, we compared and analyzed the performance of these four components in terms of JVM source code coverage.

Figure 2 shows the coverage results of *LINES*, *BRANCHES*, *FUNCTIONS* with different types of data synthesis. The test cases generated using the bug-triggering code segments synthesis method are significantly better than the method using only seeds, variables, and code blocks. This finding confirms the effectiveness of bug-triggering code segments in improving

[2]PR#17699 from the link [49]

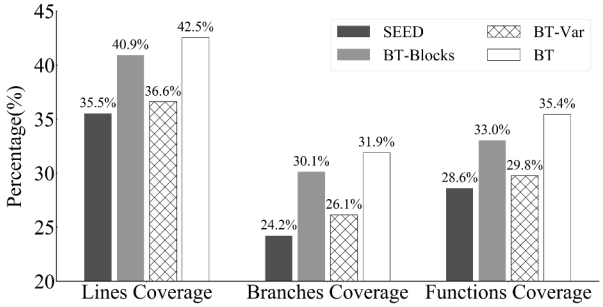


Figure 2: SEED/Var/Block/BT components coverage.

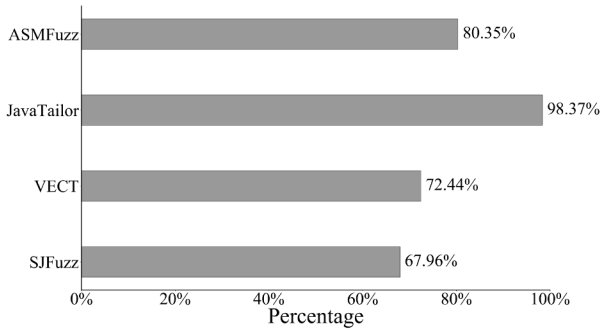


Figure 3: Comparison results of syntax/alive passing rate.

the coverage of fuzzers on JVM source code. Further analysis shows that the use of variable synthesis alone reduces the richness of test cases due to the lack of code logic diversity, while the use of code blocks alone also fails to cover deeper source code due to the lack of some essential variables. Bug-triggering code segments achieve a higher level of coverage by introducing code blocks and combining them with initialization variables, suggesting that bug-triggering code segments can improve the diversity of synthesized test cases.

In order to verify the problems posed when bug-triggering code segments are synthesized, Figure 3 shows how well ASMFuzz generates grammar for test cases with other fuzzers. The grammatical correctness of ASMFuzz is slightly lower than that of JavaTailor but higher than that of the other tools. This phenomenon is mainly because JavaTailor elaborates the syntax rules when designing test cases and filters the test cases that may fail. Whereas VECT assembles a synthesis-based retention of correct test cases to continue assembling, SJFuzz reduces the correctness rate through mutation. In contrast, ASMFuzz demonstrated more randomness in test case synthesis due to not completely filtering out the encapsulated JVM built-in classes and built-in methods, but rather retaining these elements as much as possible to challenge encapsulation and invocation of built-in methods. While this approach may have impacted the syntactic correctness of the test cases, it provides more testing opportunities to explore different branches of the source code, which may reveal more potential errors.

To evaluate the impact of combining patch extraction with fuzzing during the extraction of bug-triggering code segments, this section comparatively analyzes the extraction modules of

Authorized licensed use limited to: NORTHWESTERN POLYTECHNICAL UNIVERSITY. Downloaded on December 04,2024 at 08:55:10 UTC from IEEE Xplore. Restrictions apply.

Table IV: Coverage compare by JVM Main Function Module.

Module Name	JavaTailor	ASMFuzz	Common	JavaTailor-UNIQUE	ASMFuzz-UNIQUE	Improvement
GC	3264	**4298**	2875	389	**1423**	**▲365.8%**
Primes	601	**668**	599	2	**69**	**▲3450%**
Runtime	3190	**4092**	2099	1091	**1993**	**▲182.7%**
OOPS	3318	**3795**	2794	624	**1101**	**▲176.4%**
Total	10373	**12853**	8367	2106	**4586**	**▲217.8%**

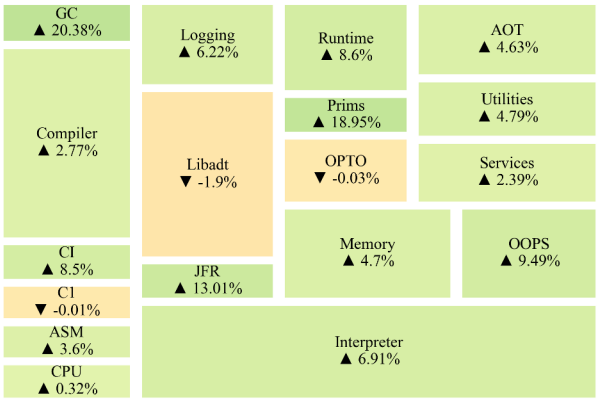


Figure 4: The line coverage improvement for each JVM component when comparing ASMFuzz with JavaTailor.

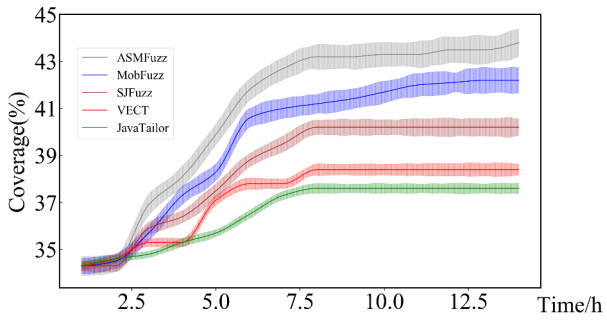


Figure 5: Comparison results of line coverage.

two tools, JavaTailor and ASMFuzz. In the experimental setup, we ensured that the seeds remained consistent and removed other components. Figure 4 compares the line coverage of two fuzzers on different modules over 24 hours on the same version of the JDK. These are core modules identified by coverage information combined with categorized tags in the patch and are representative of the majority of the source code.

The graph shows the relative rates at which ASMFuzz is higher or lower than JavaTailor coverage. ASMFuzz is very close on `C1`, `OPTO`, `Memory`, and `Utilities` modules. But especially on the `Libadt` module, JavaTailor demonstrates superior coverage. Specifically, JavaTailor dominates in the `Libadt` module. This module defines some abstract data structures at the bottom of the JVM and provides basic Accessors (`Getters`), Mutators (`Setters`), and other methods and interfaces for upper-level code. According to the module's historical bug-fixing log, no changes or updates have been made since 2021, so it is not a module of key interest for JVM testing and maintenance.

In contrast, ASMFuzz has significantly superior method coverage in the following key modules: the `GC` module responsible for garbage collection, the `Prims` module that implements the native methods in the standard library, and the Runtime module responsible for thread scheduling, lock management, compiler scheduling and support for reflection functions. These modules form the core foundation of the JDK to support the logical implementation of its main functions. This proves that ASMFuzz is also applicable to module testing beyond the JVM.

Further, Table IV shows the four most frequently called modules, and it is possible to visualize that ASMFuzz calls and covers more source code on these key modules and covers the most unique method functions (UNIQUE). `Improvement` shows the improvement of ASMFuzz over JavaTailor's coverage of source code unique. This difference mainly stems from the different distributions of bug-triggering code segments used by the two fuzzers. JavaTailor performs a comprehensive extraction and splicing of historical test cases, an approach that introduces a large number of bug-unrelated fragments. However, because developers prioritize fixes and add patches to core modules, while ASMFuzz locates bug-related fragments through patches, a collection of bug-triggering code segments that is more relevant to functional modules is constructed. This approach enhances the relevance of test cases to bugs and improves testing effectiveness on critical modules.

*3) **Effectiveness of multi-metric hybrid scheduling (RQ3)**:* In this section, to assess the effectiveness of multi-metric hybrid scheduling, we control the seed to be the same batch of generated Java programs, the bug-triggering segments dataset is acquired in a uniform way using the extraction method of this paper. We also re-implement and compare the excellent multi-objective optimization strategies in MobFuzz.

Figure 5 shows how line coverage changes as the testing time increases. As the test progresses, ASMFuzz is able to achieve a relatively high level of coverage relative to the other tools and has not yet reached a plateau during the test period. In contrast, the coverage of MobFuzz was also able to consistently increase, but the rate of coverage increase was slow. This is because the time consumed by the metrics statistics was not taken into account when scheduling the metrics combination, which resulted in the coverage statistics metrics consuming the test time, thus lowering the coverage during the test time. VECT reduces the component space and achieves better results than JavaTailor after testing for a period of time. Although the

Authorized licensed use limited to: NORTHWESTERN POLYTECHNICAL UNIVERSITY. Downloaded on December 04,2024 at 08:55:10 UTC from IEEE Xplore. Restrictions apply.

Table V: Bug-triggering ability of the comparative baselines.

	JavaTailor	SJFuzz	VECT	MobFuzz	LERE-Java	ASMFuzz
Anomalies	51	69	96	90	5	**97**
Unique	7	12	17	17	4	**20**

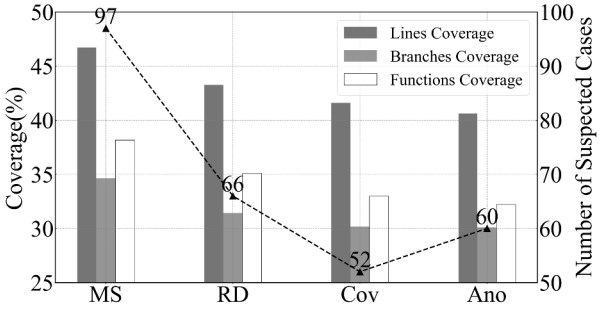


Figure 6: Compared anomalies and coverage on evaluation metrics.

component selection strategy is important, it improves slowly after reaching a higher coverage rate, because the clustering can not prioritize the use of the components that triggered the bug, which does not guarantee the continuity of the discovery of new paths. SJFuzz performs seeding and variation scheduling so that it can effectively solve the problem of single feedback. However, the variation strategy can explore more paths, but due to the increase of search space, the coverage path efficiency is lower than ASMFuzz.

Multi-metric hybrid scheduling can provide higher performance feedback. To explore the contributions of the components within ASMFuzz, we ensure that each test case has identical bug-triggering code segments and seeds. This study compares four different components of ASMFuzz: ASMFuzzMS (multi-metric scheduling), ASMFuzzRD (random scheduling), ASMFuzzAno (anomalous behavior feedback only), and ASMFuzzCov (code coverage feedback only).

As shown in Figure 6, when the execution time achieves 24 hours, ASMFuzzMS covers more source code than ASMFuzzRD, and it can be seen that the coverage of ASMFuzzCov is also lower, this is because the statistics time of coverage metrics affects the performance of fuzzing. The coverage of ASMFuzzAno is the lowest, this is because it will continue to assemble test cases for each combination, resulting in the generation of a large number of test cases that are testing the same module. However, it was observed that the number of anomalies does not match the relationship between the coverage of the other three groups and the number of suspicious cases. After detailed analysis, it was found that the anomalies return to be able to generate logically complex test cases for a particular module while generating different test cases that trigger the same test case. This shows that adding multi-metric hybrid scheduling improves values of line, branch, and method coverage. In summary, adding multi-metric hybrid scheduling feedback improves coverage and consistently covers new paths.

We designed a comparison experiment to evaluate the bug-triggering capability of ASMFuzz. Table V presents the counts of both total and unique anomalies detected by the baselines and ASMFuzz. LERE serves the program from the bug report as test cases, we reproduce it based on the original description to test JVMs, called LERE-Java. It only uncovered 4 unique anomalies. This is because conducting regression testing for each bug during the development process decreases the chances of these bugs being triggered again. Overall, the results demonstrate that ASMFuzz has a stronger bug-triggering ability.

VI. Discussion

A. Future Work

Our work designs a new code extraction technique and metric scheduling method. Moreover, we find that these methods are highly generalizable. While the bug-triggering code extraction approach demonstrates the potential for broader utilization across different compilers, it requires the ability to parse the corresponding programming language into an AST. This requirement imposes an initial adaptation step to the syntactic nuances of each new language. We have slightly adapted the extraction tool and successfully applied it to the JavaScript and Python interpreters. A better approach would be to develop a compiler extractor to accomplish these tasks, incorporating syntactic understanding functionality that would allow us to automate the processing of syntax trees from different compilers.

To address the selection of feedback metrics, the current implementation of ASMFuzz relies heavily on the computation time of the metrics statistics. This approach may not be exhaustive in capturing the multifaceted nature of fuzzing efficiency. Enriching the basis of feedback selection by integrating a range of additional factors is a future research priority. For example, incorporating metrics such as memory consumption, use case size, and execution time can significantly enhance the efficiency of selection scheduling. However, the automated addition of such enhancements is still in its infancy. In the future, more sophisticated and automated fusion algorithms should be designed to add more comprehensive influences on metrics.

B. Threats To Validity

Our work focuses on testing against the Java compiler. However, during the synthesis of test cases, code fragments may be introduced that can cause harm to the system or file system. For this reason, we use `Docker` technology to isolate the test environment from the host computer and log all

Authorized licensed use limited to: NORTHWESTERN POLYTECHNICAL UNIVERSITY. Downloaded on December 04,2024 at 08:55:10 UTC from IEEE Xplore. Restrictions apply.

system operations, making it easy to locate the exact cause of system exits due to such problems. In addition, the most recent stable release of Java is used in this paper. However, since the developers continuously update and maintain the source code, the bugs revealed need to be verified after compiling the latest updated code. We have designed separate automation scripts to automatically deploy and compile the latest release in order to obtain the latest test results.

For the specific extraction process, we select the fixed patches of the top 60% modules with the highest update frequency for locating and extracting historical test cases. To minimize this threat, we will use more metrics screening for PR to evaluate the bug-triggering code segments effect. The specific experimental threat is mainly the randomness during the experiment. To reduce the threat of randomness, we employ 5-test experiments to enhance the robustness of the experimental results. To ablate the experiment, we adapted as well as replicated it for the comparison tool. To minimize the impact of the code, we will publish together to show more details.

VII. Related Work

A. Generation Fuzzing

Generation-based fuzzing is to create test cases from scratch and there are various ways to generate test cases [50; 51; 52; 53] such as designing templates [54] and rules to generate test cases [55] based on prior knowledge. Meanwhile, the researchers generated API inputs through careful design [56; 57; 58], extracting constraints and characterization strings to express richer semantics. For example, Radamsa [59] generates malicious inputs in different formats by obtaining the characteristics of the sample files of valid data to effectively test the robustness of the software, test cases can be generated based on the rules defined in the official standard documents. DocTer [58] automatically generates diverse valid and invalid input data by analyzing the parameter constraints specified in the API standard documentation in the `DL` library for targeted testing of the API.

Generating code through model training can effectively capture code syntax and semantics, COMFORT [60] and COMFUZZ [26] utilize the `GPT` model for unsupervised learning of programming languages to generate a large number of test cases. In order to generate test cases with a higher probability of triggering software defects, Valentin Wüstholz et al. implemented the Bran [61] framework by extending the Harvey [62] framework with an on-line static analysis of the smart contract, which generates test cases capable of reaching the target location.

Compared to existing generation methods, ASMFuzz uses bug-triggering code segments to assemble test cases, which avoids the need for in-depth knowledge of bug characteristics to summarize templates, and it will not be invalidated by the initial modeling and set up program generation templates as the software version is updated.

B. Scheduling-based Fuzzing

Scheduling-based fuzzing has been widely used as a method to reduce meaningless computation. Fuzzing can be made more efficient if it is possible to select the fewest number of test cases that can trigger software bugs from a large amount of data, or the fewest number of test cases that can achieve the maximum code coverage [63]. The essence of this problem is the Minimum Set Coverage Problem (MSCP) [64], a classical `NP-hard` problem [65], which has been intensively discussed and explored by experts in the areas of seed scheduling [66; 67; 68; 69] and mutator scheduling [70; 71; 72; 73; 74; 75].

AFL [20] implements scheduling seeds when increasing code coverage. COVERSET [76] allows the selection of seed files to maximize the number of defects found in the software by designing different seed selection algorithms and using an integer linear programming problem to formalize the ex-post optimal seed selection scheduling problem. MoonLight [77]defines a cost function to measure the deviation between the seed set and the optimal coverage set and uses dynamic programming to reduce the cost function to obtain a near-optimal seed set. Confuzzion [78] introduces a variance-back orientated fuzzer on the JVM to expose types of obfuscation vulnerabilities. It uses historical execution information to generate new test cases by randomly selecting mutation methods. EvmFuzz [79] builds seed contracts with pre-set mutators and a dynamic priority scheduling algorithm to guide seed contract selection and maximize inconsistency. In this paper, we adopt ASMFuzz to schedule different feedback metrics using metrics statistics time to mitigate the impact of the lack of metrics statistics consumption in feedback metrics scheduling and efficiently utilize the execution information and coverage information.

VIII. conclusion

We present ASMFuzz, a new fuzzing technique designed to detect defects in Java compilers. It synthesizes bug-related test cases and employs a multi-guide metric feedback scheduling strategy to discover unknown defects while preventing premature test convergence. ASMFuzz combines patches with historical test cases to identify defect-related fragments that contain critical defect trigger information. To handle a wide range of test cases and prevent a rapid coverage plateau, we use two feedback metrics designed to capture more useful test information. In addition, we manage the feedback frequency using a linear confidence upper bound to minimize statistical delays in fuzzing. A working prototype of ASMFuzz has been tested on major compiler versions including HotSpot, OpenJ9, and GraalVM, successfully identifying and confirming 16 new bugs. These results highlight the capability of ASMFuzz to discover additional bugs and enable comprehensive testing of important modules.

Acknowledgment

We thank the anonymous reviewers for their insightful feedback. This work was supported in part by the National Natural Science Foundation of China (NSFC) under grant agreements 62102315, and 62372373, the China Postdoctoral Science Foundation Fellowship (2022M712575), the Shaanxi International Science and Technology Cooperation Program (2023-GHZD-04), the Shaanxi Province “Engineers + Scientists" Team Building Program (2023KXJ-055).

Authorized licensed use limited to: NORTHWESTERN POLYTECHNICAL UNIVERSITY. Downloaded on December 04,2024 at 08:55:10 UTC from IEEE Xplore. Restrictions apply.

REFERENCES

[1] "hotspot," https://openjdk.org/groups/hotspot/.
[2] "Graalvm," https://www.graalvm.org/.
[3] "Openj9," https://www.eclipse.org/openj9/.
[4] "Jikesrvm," https://github.com/JikesRVM/JikesRVM/.
[5] "Azulzulu," https://www.azulsystems.com/products/zulu/, n.d.
[6] "Azuzing," https://www.azulsystems.com/products/zing/, n.d.
[7] B. Liu, G. Meng, W. Zou, Q. Gong, F. Li, M. Lin, D. Sun, W. Huo, and C. Zhang, "A large-scale empirical study on vulnerability distribution within projects and the lessons learned," in *2020 IEEE/ACM 42nd International Conference on Software Engineering (ICSE)*. IEEE, 2020, pp. 1547–1559.
[8] L. Luo, "Software testing techniques," *Institute for software research international Carnegie mellon university Pittsburgh, PA*, vol. 15232, no. 1-19, p. 19, 2001.
[9] S. U. Farooq, "Evaluating effectiveness of software testing techniques with emphasis on enhancing software reliability," *Journal of emerging trends in Computing and Information Sciences*, vol. 2, no. 12, 2012.
[10] S. U. Farooq and S. Quadri, "Empirical evaluation of software testing techniques–need, issues and mitigation," *Software engineering: an international Journal*, vol. 3, no. 3, pp. 41–51, 2013.
[11] Y. Chen, T. Su, C. Sun, Z. Su, and J. Zhao, "Coverage-directed differential testing of jvm implementations," in *proceedings of the 37th ACM SIGPLAN Conference on Programming Language Design and Implementation*, 2016, pp. 85–99.
[12] Y. Chen, T. Su, and Z. Su, "Deep differential testing of jvm implementations," in *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*. IEEE, 2019, pp. 1257–1268.
[13] Y. Zhao, Z. Wang, J. Chen, M. Liu, M. Wu, Y. Zhang, and L. Zhang, "History-driven test program synthesis for jvm testing," in *2022 IEEE/ACM 44th International Conference on Software Engineering (ICSE)*. IEEE, 2022, pp. 1133–1144.
[14] M. Wu, Y. Ouyang, M. Lu, J. Chen, Y. Zhao, H. Cui, Y. Guo, and Y. Zhang, "Sjfuzz: Seed & mutator scheduling for jvm fuzzing," in *2023 The ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, 2023.
[15] T. Gao, J. Chen, Y. Zhao, Y. Zhang, and L. Zhang, "Vectorizing program ingredients for better jvm testing," in *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2023, pp. 526–537.
[16] V. Vikram, I. Laybourn, A. Li, N. Nair, K. OBrien, R. Sanna, and R. Padhye, "Guiding greybox fuzzing with mutation testing," in *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2023, pp. 929–941.
[17] K. Wang, Y. Wang, J. Wang, and Q. Wang, "Fuzzing with sequence diversity inference for sequential decision-making model testing," in *2023 IEEE 34th International Symposium on Software Reliability Engineering (ISSRE)*. IEEE, 2023, pp. 706–717.
[18] H. Zhong, "Enriching compiler testing with real program from bug report," in *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*, 2022, pp. 1–12.
[19] W. Gao, V.-T. Pham, D. Liu, O. Chang, T. Murray, and B. I. Rubinstein, "Beyond the coverage plateau: A comprehensive study of fuzz blockers (registered report)," in *Proceedings of the 2nd International Fuzzing Workshop*, 2023, pp. 47–55.
[20] M. Zalewski, "American fuzzy lop," https://lcamtuf.coredump.cx/afl/, 2014.
[21] A. Groce, R. van Tonder, G. T. Kalburgi, and C. Le Goues, "Making no-fuss compiler fuzzing effective," in *Proceedings of the 31st ACM SIGPLAN International Conference on Compiler Construction*, 2022, pp. 194–204.
[22] Y. Li, Y. Xue, H. Chen, X. Wu, C. Zhang, X. Xie, H. Wang, and Y. Liu, "Cerebro: context-aware adaptive fuzzing for effective vulnerability detection," in *Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2019, pp. 533–544.
[23] G. Zhang, P. Wang, T. Yue, X. Kong, S. Huang, X. Zhou, and K. Lu, "Mobfuzz: Adaptive multi-objective optimization in gray-box fuzzing," *arXiv preprint arXiv:2401.15956*, 2024.
[24] R. Hosseini and P. Brusilovsky, "Javaparser: A fine-grain concept indexing tool for java problems," in *CEUR Workshop Proceedings*, vol. 1009. University of Pittsburgh, 2013, pp. 60–63.
[25] L. Li, W. Chu, J. Langford, and R. E. Schapire, "A contextual-bandit approach to personalized news article recommendation," in *Proceedings of the 19th international conference on World wide web*, 2010, pp. 661–670.
[26] G. Ye, T. Hu, Z. Tang, Z. Fan, S. H. Tan, B. Zhang, W. Qian, and Z. Wang, "A generative and mutational approach for synthesizing bug-exposing test cases to guide compiler fuzzing," in *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. Association for Computing Machinery (ACM), 2023.
[27] C. Holler, K. Herzig, and A. Zeller, "Fuzzing with code fragments," in *Proceedings of the 21st USENIX Security Symposium (USENIX Security)*, 2012, pp. 445–458.
[28] S. Park, W. Xu, I. Yun, D. Jang, and T. Kim, "Fuzzing javascript engines with aspect-preserving mutation," in *Proceedings of the 41st IEEE Symposium on Security and Privacy (S&P)*, 2020, pp. 1629–1642.
[29] S. Lee, H. Han, S. K. Cha, and S. Son, "Montage: A neural network language model-guided javascript fuzzer," in *Proceedings of the 29th USENIX Security Symposium (USENIX Security)*. USENIX, 2020, pp. 2613–2630.
[30] H. Han, D. Oh, and S. K. Cha, "Codealchemist: Semantics-aware code generation to find vulnerabilities in javascript engines," in *Proceedings of the 26th Annual Network and Distributed System Security Symposium (NDSS)*, 2019.
[31] J. Chen, J. Patra, M. Pradel, Y. Xiong, H. Zhang, D. Hao, and L. Zhang, "A survey of compiler testing," *ACM Computing Surveys (CSUR)*, vol. 53, no. 1, pp. 1–36, 2020.
[32] J. Chen, W. Hu, D. Hao, Y. Xiong, H. Zhang, L. Zhang, and B. Xie, "An empirical comparison of compiler testing techniques," in *Proceedings of the 38th International Conference on Software Engineering*, 2016, pp. 180–190.
[33] J. Chen and C. Suo, "Boosting compiler testing via compiler optimization exploration," *ACM Transactions on Software Engineering and Methodology (TOSEM)*, vol. 31, no. 4, pp. 1–33, 2022.
[34] F. Bazzichi and I. Spadafora, "An automatic generator for compiler testing," *IEEE Transactions on Software Engineering*, no. 4, pp. 343–353, 1982.
[35] A. S. Kossatchev and M. Posypkin, "Survey of compiler testing methods," *Programming and Computer Software*, vol. 31, pp. 10–19, 2005.
[36] Y. Tang, Z. Ren, W. Kong, and H. Jiang, "Compiler testing: a systematic literature analysis," *Frontiers of Computer Science*, vol. 14, pp. 1–20, 2020.
[37] C. Lyu, S. Ji, X. Zhang, H. Liang, B. Zhao, K. Lu, and R. Beyah, "Ems: History-driven mutation for coverage-based fuzzing," in *29th Annual Network and Distributed System Security Symposium. https://dx. doi. org/10.14722/ndss*, vol. 10, 2022.
[38] M. Böhme, V. J. Manès, and S. K. Cha, "Boosting fuzzer efficiency: An information theoretic perspective," in *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2020, pp. 678–689.
[39] R. Feldt, S. Poulding, D. Clark, and S. Yoo, "Test set diameter: Quantifying the diversity of sets of test cases," in *2016 IEEE international conference on software testing, verification and validation (ICST)*. IEEE, 2016, pp. 223–233.
[40] N. Alshahwan and M. Harman, "Coverage and fault detection of the output-uniqueness test selection criteria," in *Proceedings of the 2014 International Symposium on Software Testing and Analysis*, 2014, pp. 181–192.
[41] "Copymemory.java," https://github.com/openjdk/jdk11u-dev/blob/70862e011ba642a782cf28004d2f9dd807971788/test/jdk/sun/misc/CopyMemory.java#L66C1-L68C10.
[42] "Test7190310_unsafe.java," https://github.com/openjdk/jdk11u-dev/blob/70862e011ba642a782cf28004d2f9dd807971788/test/hotspot/jtreg/compiler/c2/Test7190310_unsafe.java#L61C1-L62C1.
[43] "Gcov," https://gcc.gnu.org/onlinedocs/gcc/Gcov.html.
[44] N. Hinds, P. Larson, H. Franke, and M. Ridgeway, "Using code coverage tools in the linux kernel," *ACM SIGSOFT Software Engineering Notes (ACM'04)*, pp. 70–81, 2014.
[45] "Indify.java," https://github.com/openjdk/jdk/blob/564a72e1dba0f145600c8e7eff66992fbf294df0/test/jdk/java/lang/invoke/indify/Indify.java#L788.
[46] "Jdk-7001423," https://bugs.openjdk.org/browse/JDK-7001423.
[47] "Inputargument.java," https://github.com/openjdk/jdk/blob/9c7a6eabb93c570fdb74076edc931576ed6be3e0/test/jdk/java/lang/management/RuntimeMXBean/InputArgument.java#L79.
[48] "Jdk-319a3b," https://github.com/openjdk/jdk/commit/319a3b994703aac84df7bcde272adfcb3cdbbbf0.
[49] "Pr#17699," https://github.com/eclipse-openj9/openj9/pull/17699/files.
[50] O. Bastani, R. Sharma, A. Aiken, and P. Liang, "Synthesizing program input grammars," *ACM SIGPLAN Notices*, vol. 52, no. 6, pp. 95–110,

Authorized licensed use limited to: NORTHWESTERN POLYTECHNICAL UNIVERSITY. Downloaded on December 04,2024 at 08:55:10 UTC from IEEE Xplore. Restrictions apply.

2017.
[51] P. Godefroid, A. Kiezun, and M. Y. Levin, "Grammar-based whitebox fuzzing," in *Proceedings of the 29th ACM SIGPLAN conference on programming language design and implementation*, 2008, pp. 206–215.
[52] F. Molina, M. d'Amorim, and N. Aguirre, "Fuzzing class specifications," in *Proceedings of the 44th International Conference on Software Engineering*, 2022, pp. 1008–1020.
[53] P. Srivastava and M. Payer, "Gramatron: Effective grammar-aware fuzzing," in *Proceedings of the 30th acm sigsoft international symposium on software testing and analysis*, 2021, pp. 244–256.
[54] J. Hua and S. Khurshid, "Edsketch: Execution-driven sketching for java," in *Proceedings of the 24th ACM SIGSOFT International SPIN Symposium on Model Checking of Software*, 2017, pp. 162–171.
[55] P. Bielik, V. Raychev, and M. Vechev, "Phog: probabilistic model for code," in *International conference on machine learning*. PMLR, 2016, pp. 2933–2942.
[56] M. Wang, B. Cui, J. Yan, J. Yan, and J. Zhang, "String test data generation for java programs," in *2022 IEEE 33rd International Symposium on Software Reliability Engineering (ISSRE)*. IEEE, 2022, pp. 251–262.
[57] T. Hu, G. Ye, Z. Tang, S. H. Tan, H. Wang, M. Li, and Z. Wang, "Upbeat: Test input checks of q# quantum libraries," in *Proceedings of ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*. ACM, 2024.
[58] D. Xie, Y. Li, M. Kim, H. V. Pham, L. Tan, X. Zhang, and M. W. Godfrey, "Docter: documentation-guided fuzzing for testing deep learning api functions," in *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2022, pp. 176–188.
[59] "Radamsa," https://gitlab.com/akihe/radamsa.
[60] G. Ye, Z. Tang, S. H. Tan, S. Huang, D. Fang, X. Sun, L. Bian, H. Wang, and Z. Wang, "Automated conformance testing for javascript engines via deep compiler fuzzing," in *Proceedings of the 42nd ACM SIGPLAN international conference on programming language design and implementation*, 2021, pp. 435–450.
[61] V. Wüstholz and M. Christakis, "Harvey: A greybox fuzzer for smart contracts," in *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2020, pp. 1398–1409.
[62] ——, "Targeted greybox fuzzing with static lookahead analysis," in *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering*, 2020, pp. 789–800.
[63] M. Ivanković, G. Petrović, R. Just, and G. Fraser, "Code coverage at google," in *Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2019, pp. 955–963.
[64] C. Lecat, C. M. Li, C. Lucet, and Y. Li, "Exact methods for the minimum sum coloring problem," 2015.
[65] D. S. Hochba, "Approximation algorithms for np-hard problems," *ACM Sigact News*, vol. 28, no. 2, pp. 40–52, 1997.
[66] M. Böhme, V.-T. Pham, M.-D. Nguyen, and A. Roychoudhury, "Directed greybox fuzzing," in *Proceedings of the 2017 ACM SIGSAC conference on computer and communications security*, 2017, pp. 2329–2344.
[67] K. Zhang, X. Xiao, X. Zhu, R. Sun, M. Xue, and S. Wen, "Path transitions tell more: Optimizing fuzzing schedules via runtime program states," in *Proceedings of the 44th International Conference on Software Engineering*, 2022, pp. 1658–1668.
[68] R. Qian, Q. Zhang, C. Fang, D. Yang, S. Li, B. Li, and Z. Chen, "Dipri: Distance-based seed prioritization for greybox fuzzing," *ACM Transactions on Software Engineering and Methodology*, 2024.
[69] S. Luo, A. Herrera, P. Quirk, M. Chase, D. C. Ranasinghe, and S. S. Kanhere, "Make out like a (multi-armed) bandit: Improving the odds of fuzzer seed scheduling with t-scheduler," *arXiv preprint arXiv:2312.04749*, 2023.
[70] Z. Gao, H. Xiong, W. Dong, R. Chang, R. Yang, Y. Zhou, and L. Jiang, "Fa-fuzz: A novel scheduling scheme using firefly algorithm for mutation-based fuzzing," *IEEE Transactions on Software Engineering*, 2023.
[71] D. Blackwell and D. Clark, "Prescientfuzz: A more effective exploration approach for grey-box fuzzing," *arXiv preprint arXiv:2404.18887*, 2024.
[72] L. Jiang, H. Yuan, M. Wu, L. Zhang, and Y. Zhang, "Evaluating and improving hybrid fuzzing," in *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 2023, pp. 410–422.
[73] M. Wu, K. Chen, Q. Luo, J. Xiang, J. Qi, J. Chen, H. Cui, and Y. Zhang, "Enhancing coverage-guided fuzzing via phantom program," in *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2023, pp. 1037–1049.
[74] M. Wu, M. Lu, H. Cui, J. Chen, Y. Zhang, and L. Zhang, "Jitfuzz: Coverage-guided fuzzing for jvm just-in-time compilers," in *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 2023, pp. 56–68.
[75] C. Kong and F. Zhang, "A fuzzing method to reduce the number of generated invalid testcases," in *2022 2nd Conference on High Performance Computing and Communication Engineering (HPCCE 2022)*, vol. 12605. SPIE, 2023, pp. 202–208.
[76] A. Rebert, S. K. Cha, T. Avgerinos, J. Foote, D. Warren, G. Grieco, and D. Brumley, "Optimizing seed selection for fuzzing," in *23rd USENIX Security Symposium (USENIX Security 14)*, 2014, pp. 861–875.
[77] "Moonlight," https://github.com/moonlight-project/moonlight.
[78] W. Bonnaventure, A. Khanfir, A. Bartel, M. Papadakis, and Y. Le Traon, "Confuzzion: A java virtual machine fuzzer for type confusion vulnerabilities," in *2021 IEEE 21st International Conference on Software Quality, Reliability and Security (QRS)*. IEEE, 2021, pp. 586–597.
[79] Y. Fu, M. Ren, F. Ma, X. Yang, H. Shi, S. Li, and X. Liao, "Evmfuzz: Differential fuzz testing of ethereum virtual machine," *Journal of Software: Evolution and Process*, vol. 36, no. 4, p. e2556, 2024.

Authorized licensed use limited to: NORTHWESTERN POLYTECHNICAL UNIVERSITY. Downloaded on December 04,2024 at 08:55:10 UTC from IEEE Xplore. Restrictions apply.